

Matching Algorithm Research Note

2026-07-02

ABSTRACT

This note reformulates the post-purchase advertising problem as a contextual-bandit ad filtering problem for an in-house demand-side platform (DSP). The objective is not yet to solve real-time bidding end-to-end, but to construct a principled first layer that selects a small set of promising ads from a large candidate pool under exploration, cold-start, delayed feedback, and auction-induced selection bias. We define the decision process, reward, logging requirements, offline evaluation protocol, and compare multiple contextual-bandit algorithms: Disjoint LinUCB, Hybrid LinUCB, Linear Thompson Sampling, Logistic/GLM Bandits, and Neural Bandits. The recommended first implementation is a feature-based LinUCB/LinTS baseline with inverse-propensity-aware logging and a clear path to hybrid and generalized-linear extensions.

Scope

The product surface is a post-purchase page in a car-rental booking service. A user has just completed a reservation, so the system observes contextual variables such as car type, area, rental duration, price, device, and possibly session history. The DSP owns a set of ads, each with metadata such as category, target area, target car type, advertiser, budget, and cold-start status. The filtering layer chooses a small set of ads to pass to the bidding layer. The bidding layer then decides bid prices using value prediction, pacing, and auction logic.

This document focuses on the filtering layer. It deliberately separates filtering from bidding because RTB contains censored feedback, budget constraints, pacing, and strategic auction effects that are not fully captured by a standard contextual bandit.

Decision Problem

Objects

At interaction $t=1,2,\dots,T$:

- * $c_t \in \mathcal{C}$ is the observed purchase context: car type, area, area type, rental days, price bucket, device, session-level summaries, and time features.
- * $A_t = \{a_{t1}, \dots, a_{tN_t}\}$ is the eligible ad set after hard business filters such as active campaign, area eligibility, creative approval, and budget availability.
- * $z(a) \in \mathcal{Z}$ denotes ad-side features: category, target area, target car type, advertiser, campaign age, novelty flag, and historical priors if available.
- * $x_t(a) = (c_t, z(a))^d$ is a joint context--ad feature vector.
- * The filtering policy selects a subset $S_t \subseteq A_t$ with $|S_t|=K$; the bidding layer receives S_t .

Reward

Let $Y_t(a)$ be the potential outcome for ad a in context c_t . Depending on the product goal, the reward can be one of:

$r_t(a) \in \{\text{click}, \text{value-converted}, \text{value-clicked}, \text{value-won}\}$
+ cvrv_conversion
- cost .

For the first filtering-layer prototype, click reward is acceptable because it is frequent and timely. For production optimization, the reward should move toward value-per-impression or value-per-won-impression after accounting for delayed conversions and auction costs.

The ideal objective for top-K filtering is

$$S_t^* = \arg\max_{S_t \subseteq A_t, |S_t|=K} \sum_{a \in S_t} Y_t(a)$$

Equivalently, the regret against an oracle that knows expected rewards is

$$R_T = \sum_{t=1}^T (R_t^* - R_t)$$

$$(\sum_a S_t^a(c_t, a) - \sum_a S_t^a(c_t, a)),$$

where $(c, a) =$ and S_t^a is the top-K set by .

Feature Design

A linear first model requires interactions to be explicit. A minimal feature map is:

$x_t(a) =$.

The feature map should be versioned. Every logged decision must store the feature version, eligible ad set, chosen set, ranking scores, and action probabilities or propensities.

Feedback and Bias

In RTB, feedback is censored. The system observes clicks or conversions only for ads that were selected, bid on, won, and rendered. Therefore, naive training on won impressions can learn the behavior of the auction and pacing system rather than intrinsic ad relevance.

A production-quality log should contain:

- * request id, context, timestamp, user/session privacy-safe keys;
- * all eligible ads before filtering;
- * selected top-K set, scores, and propensities;
- * bid decision, bid price, pacing multiplier, and shading multiplier;
- * win/loss, clearing price when available, impression, click, conversion, and delayed labels;
- * model version, feature version, and exploration parameters.

For offline evaluation, use randomized exploration traffic and estimators such as inverse propensity scoring (IPS), self-normalized IPS, or doubly robust estimators. Deterministic logs from a greedy policy are insufficient for unbiased evaluation of alternative policies.

Algorithm Candidates

Disjoint LinUCB

Classical disjoint LinUCB maintains independent parameters per arm. For arm a :

$$\begin{aligned} A_a &| = I + \sum_{s: a_s=a} x_s x_s^T, \\ b_a &| = \sum_{s: a_s=a} r_s x_s, \\ \hat{\mu}_a &| = A_a^{-1} b_a. \end{aligned}$$

The score is

$$p_t(a) = \hat{\mu}_a^T x_t(a) + x_t(a)^T A_a^{-1} x_t(a).$$

It is easy to implement and interpretable, but pure per-arm learning is weak for new ads because a newly introduced arm has no history.

Feature-Based Shared LinUCB

A practical first step is to represent ads by features and learn one shared model:

$$\begin{aligned} A &| = I + \sum x_s x_s^T, \\ b &| = \sum r_s x_s, \\ \hat{\mu} &| = A^{-1} b, \\ p_t(a) &| = \hat{\mu}^T x_t(a) + x_t(a)^T A^{-1} x_t(a). \end{aligned}$$

This transfers learning across ads through metadata and interactions, so it is better for cold start. It should be the first reliable baseline for this repository.

Hybrid LinUCB

Hybrid LinUCB decomposes the reward into a global component and an arm-specific component:

$$z_t(a) + x_t(a)^T \hat{\mu}_a.$$

Here $z_t(a)$ captures common features shared across arms, while $x_t(a)$ captures arm-local

features. This is appropriate once there is enough data to justify advertiser- or ad-level idiosyncratic effects without losing cold-start generalization.

Linear Thompson Sampling

Linear Thompson Sampling uses the same Bayesian linear-regression structure but samples a parameter vector instead of adding an optimism bonus:

$$\begin{aligned} \mu &= A^{-1}b, \\ \mu_t &| N(\mu, \sigma^2 A^{-1}), \\ p_t(a) &= \mu^T x_t(a). \end{aligned}$$

It often behaves more smoothly than UCB in production ranking systems. Because it shares most linear-algebra machinery with LinUCB, it should be implemented as the main comparison baseline.

Logistic or GLM Bandits

Click and conversion are binary, so a logistic mean function is more natural than a linear mean:

$$\begin{aligned} (r=1 \mid x, a) &= \sigma(\mu^T x), \\ \mu &= \frac{1}{1 + e^{-u}}. \end{aligned}$$

GLM bandits are appropriate if linear predictions become poorly calibrated, especially for low CTR regimes. They are more complex because online confidence bounds require curvature or approximate posterior updates.

Neural Bandits

Neural bandits replace the hand-engineered linear feature map with a neural representation $f(c, a)$. They can capture nonlinear interactions, but they require more data, careful uncertainty estimation, and stronger monitoring. They are not recommended as the first production algorithm.

Comparison Matrix

Method | Strengths | Weaknesses | Recommended role

Disjoint LinUCB | Simple, interpretable, per-ad parameters | Poor cold start; many ads imply many matrices | Educational baseline only

Shared LinUCB | Simple; fast; transfers across ads; easy cold start | Linear assumption; feature engineering required | First implementation

Hybrid LinUCB | Balances global transfer and arm-specific effects | More bookkeeping; more parameters | Second-stage extension

Linear Thompson Sampling | Smooth exploration; shares LinUCB state | Requires sampling and variance calibration | Main A/B comparison with LinUCB

Logistic/GLM Bandit | Better for binary rewards and calibration | More complex online inference | Use after linear baseline diagnostics

Neural Bandit | Nonlinear representation power | Data hungry; uncertainty is hard | Later-stage research track

Recommended Implementation Plan

- * Define production-grade schemas for contexts, ads, eligible sets, actions, propensities, and outcomes.
- * Implement a shared linear model abstraction with Sherman--Morrison updates for A^{-1} .
- * Implement both LinUCB and Linear Thompson Sampling on the same feature store.
- * Add semi-bandit top-K learning: update the model for each displayed ad with observed reward.
- * Add randomized logging traffic and propensity recording before claiming offline gains.
- * Evaluate with reward@K, CTR@K, cumulative regret in simulation, IPS/SNIPS on logs, and calibration plots.
- * Introduce Hybrid LinUCB only after the shared model has stable logs and enough per-ad observations.
- * Keep the bidding layer separate: expected value from the filtering layer should become one input to bid price, together with pacing, budget, and bid shading.

References

*

L. Li, W. Chu, J. Langford, and R. E. Schapire.
A Contextual-Bandit Approach to Personalized News Article Recommendation.
WWW, 2010.

*

W. Chu, L. Li, L. Reyzin, and R. E. Schapire.
Contextual Bandits with Linear Payoff Functions.
AISTATS, 2011.

*

S. Agrawal and N. Goyal.
Thompson Sampling for Contextual Bandits with Linear Payoffs.
ICML, 2013.

*

L. Li, W. Chu, J. Langford, and X. Wang.
Unbiased Offline Evaluation of Contextual-Bandit-based News Article Recommendation Algorithms.
WSDM, 2011.

*

J. Wang, W. Zhang, and S. Yuan.
Display Advertising with Real-Time Bidding (RTB) and Behavioural Targeting.
Foundations and Trends in Information Retrieval, 2017.

*

W. Zhang, S. Yuan, and J. Wang.
Optimal Real-Time Bidding for Display Advertising.
KDD, 2014.

*

J. Xu, K.-c. Lee, W. Li, Q. Qi, and Q. Lu.
Smart Pacing for Effective Online Ad Campaign Optimization.
KDD, 2015.

*

W. Zhang, T. Zhou, J. Wang, and J. Xu.
Bid-aware Gradient Descent for Unbiased Learning with Censored Data in Display Advertising.
KDD, 2016.

